

ALGORITMOS, PSEUDOCÓDIGO Y DIAGRAMAS DE FLUJO

Guía didáctica de PSeInt para 1.º de DAW



Rubén Pérez Ibáñez

IES Álvaro Falomir · Curso 2026/27

Módulo: Programación

Esta guía está diseñada para aprender algoritmia mediante la práctica. Cada concepto se introduce a partir de un problema sencillo, se expresa después en pseudocódigo y, cuando ayuda a comprender el flujo de ejecución, se acompaña de un diagrama. El objetivo no es memorizar palabras clave ni símbolos aislados, sino aprender a descomponer problemas, ordenar operaciones, tomar decisiones, controlar repeticiones y comprobar que una solución termina y produce el resultado esperado.

Enfoque de trabajo. Cada apartado sigue, siempre que sea posible, la secuencia problema → idea → pseudocódigo → prueba → diagrama → ejercicio. PSeInt se utiliza como puente hacia Java, no como un fin en sí mismo.

Cómo utilizar esta guía

- Lee el enunciado y piensa primero qué datos entran, qué proceso hay que realizar y qué resultado debe salir.
- Escribe una primera solución en lenguaje natural o por pasos antes de preocuparte por la sintaxis.
- Pasa esa solución a PSeInt y ejecútala con varios juegos de datos, incluidos casos límite.
- Utiliza los diagramas de flujo cuando quieras visualizar decisiones, repeticiones o la unión de varios caminos.
- Realiza las trazas a mano: son una herramienta esencial para entender el estado de las variables.

Índice de contenidos

1. Pensar antes de programar
2. Algoritmos: definición y propiedades
3. Entrada, proceso y salida
4. Pseudocódigo y PSeInt
5. Datos, variables, constantes y tipos
6. Expresiones y operadores
7. Escribir, Leer y asignar
8. Decisiones: Si, SiNo y Segun
9. Repeticiones: Mientras, Repetir y Para
10. Contadores, acumuladores y banderas
11. Diagramas de flujo y ordinogramas
12. Trazas y depuración
13. Ejemplos resueltos paso a paso
14. Banco de ejercicios
15. Referencia rápida y recursos

1. Pensar antes de programar

Programar no consiste en escribir instrucciones al azar hasta que el programa parece funcionar. Antes de utilizar el teclado conviene transformar el problema en una secuencia de pasos suficientemente precisa como para que otra persona pueda seguirla sin tener que improvisar.

1.1. Del problema al algoritmo

Una forma práctica de comenzar consiste en responder cuatro preguntas: ¿qué se pide?, ¿qué datos conocemos o debe introducir el usuario?, ¿qué resultado debe producirse? y ¿qué operaciones, decisiones o repeticiones permiten transformar las entradas en esa salida?

1. Comprender el enunciado y eliminar ambigüedades.
2. Identificar entradas y salidas.
3. Descomponer la solución en pasos pequeños.
4. Localizar decisiones y repeticiones.
5. Comprobar el algoritmo con ejemplos antes de programarlo.

Regla de clase. Si no puedes explicar con palabras qué debe hacer el algoritmo, todavía no es buen momento para escribir código.

1.2. Ejemplo sin código

Problema: calcular el precio final de una compra aplicando un descuento del 10 % cuando el importe supera 100 euros.

Parte	Contenido
Entrada	importe de la compra
Proceso	comprobar si importe > 100; si se cumple, calcular descuento
Salida	precio final

En lenguaje natural: leer el importe; si supera 100, calcular el 90 % del importe; en caso contrario mantenerlo; finalmente mostrar el precio final.

2. Algoritmos: definición y propiedades

Un algoritmo es una secuencia ordenada y finita de pasos que permite resolver un problema. El orden es esencial: cada paso ocupa una posición concreta y cumple una función. El algoritmo debe expresar con claridad qué hacer, en qué orden y bajo qué condiciones, sin depender de interpretaciones implícitas.

2.1. Propiedades fundamentales

Propiedad	Qué significa	Cómo comprobarla
Preciso	Los pasos están expresados sin ambigüedad y en un orden claro.	Dos personas deberían interpretar la misma acción.
Definido	Con los mismos datos de entrada, el procedimiento sigue las mismas reglas.	Repite la prueba con la misma entrada.
Finito	El procedimiento termina tras un número limitado de pasos.	Revisa especialmente las condiciones de los bucles.
Fiable	Produce resultados correctos para los casos previstos.	Prueba casos normales, límite y erróneos.
Robusto	Prevé entradas o situaciones problemáticas.	Valida datos y evita operaciones imposibles.
Eficiente	Evita trabajo y recursos innecesarios.	Compara soluciones y elimina pasos redundantes.
Legible	Se entiende con facilidad y puede mantenerse.	Usa nombres descriptivos, sangría y comentarios útiles.

2.2. Correcto no significa solo “funciona una vez”

Una solución que funciona con el ejemplo utilizado en clase puede fallar con cero, con números negativos, con valores límite o con una entrada inesperada. Por ello, diseñar un algoritmo también implica prever casos de prueba y comprobar su comportamiento en situaciones diferentes.

Ejemplo. Si calculas una media, debes pensar qué ocurre cuando la cantidad de valores es cero antes de realizar una división.

3. Entrada, proceso y salida

Muchos problemas iniciales se comprenden mejor si se separan en tres partes: los datos que recibe el algoritmo, las operaciones que realiza con ellos y la información que devuelve. Esta organización se resume mediante el modelo Entrada - Proceso - Salida (EPS).



Figura 1. Modelo Entrada - Proceso - Salida.

3.1. Ejemplos de EPS

Problema	Entrada	Proceso	Salida
Área de un rectángulo	base, altura	base * altura	área
Media de tres notas	n1, n2, n3	$(n1+n2+n3)/3$	media
Par o impar	n	$n \text{ MOD } 2$	mensaje par/impar
Mayor de dos valores	a, b	comparar a y b	mayor o igualdad

Idea importante. La salida no tiene por qué ser únicamente un número. También puede ser un mensaje, un valor lógico o varios resultados.

4. Pseudocódigo y PSeInt

El pseudocódigo permite describir un algoritmo mediante instrucciones cercanas al lenguaje natural, sin depender todavía de la sintaxis de un lenguaje de programación concreto. En esta guía utilizaremos PSeInt porque permite escribir, ejecutar y depurar pseudocódigo en español, además de visualizar su representación mediante diagramas de flujo.

4.1. Estructura mínima

```
Algoritmo HolaMundo
    Escribir "Hola mundo";
FinAlgoritmo
```

Algoritmo marca el comienzo de la solución; el nombre permite identificarla; Escribir genera una salida; y FinAlgoritmo indica su final. La sangría no modifica por sí sola el resultado, pero resulta fundamental para distinguir visualmente los bloques y hacer el código más legible.

4.2. Pseudocódigo genérico y sintaxis de PSeInt

El pseudocódigo no tiene una sintaxis universal. Distintos materiales y herramientas pueden expresar la misma estructura con palabras o convenciones diferentes. En esta asignatura utilizaremos de forma coherente la sintaxis aceptada por PSeInt para evitar ambigüedades y facilitar el paso posterior a Java.

Idea	En esta guía / PSeInt
Inicio y fin	Algoritmo nombre ... FinAlgoritmo
Salida	Escribir ...
Entrada	Leer variable;
Asignación	variable <- expresión;
Condición	Si condición Entonces ... FinSi
Alternativa	Si ... Entonces ... SiNo ... FinSi
Bucle condicional	Mientras condición Hacer ... FinMientras
Bucle poscondicional	Repetir ... Hasta Que condición
Bucle contado	Para i <- inicio Hasta fin Con Paso p Hacer ... FinPara
Comentario	// comentario

Atención. No copies literalmente una notación de pseudocódigo de otra fuente y esperes que PSeInt la ejecute. Traduce siempre la idea a la sintaxis de trabajo de clase.

4.3. PSeInt Web

La versión web de PSeInt permite escribir y ejecutar los ejemplos directamente desde el navegador. En la web del curso se incluyen, además, ejercicios autocorregibles, la guía de consulta y enlaces a las herramientas utilizadas en clase.

5. Datos, variables, constantes y tipos

Un programa trabaja con datos. Algunos valores permanecen fijos durante la ejecución y otros cambian a medida que avanza el algoritmo. Una variable es un espacio identificado por un nombre cuyo contenido puede modificarse; una constante representa un valor que, conceptualmente, permanece invariable durante la solución.

5.1. Variables

```
Definir edad Como Entero;
Definir precio Como Real;
Definir nombre Como Caracter;
Definir encontrado Como Logico;
```

En los primeros ejercicios conviene elegir nombres que describan el contenido de cada variable, como edad, notaMedia o precioFinal. Evita identificadores genéricos si el problema permite utilizar un nombre más significativo. No uses espacios y mantén un criterio de escritura coherente durante todo el algoritmo.

5.2. Tipos básicos

Tipo	Uso típico	Ejemplo
Entero	números sin parte decimal	18, -3, 250
Real	números con decimales	7.5, 3.1416
Caracter	texto o cadenas de caracteres	"Rubén", "DAW"
Logico	condiciones	Verdadero / Falso

5.3. El valor de una variable cambia

```
Definir contador Como Entero;
contador <- 0;
contador <- contador + 1;
Escribir contador;
```

La segunda asignación no representa una igualdad matemática. Primero se evalúa la expresión contador + 1 utilizando el valor actual de contador; después, el resultado obtenido sustituye al valor anterior almacenado en la variable.

Error frecuente. Confundir = (comparación en una condición) con <- (asignación de un valor a una variable).

6. Expresiones y operadores

Una expresión combina valores, variables y operadores para obtener un resultado. Según los elementos que intervienen, las expresiones pueden ser aritméticas, relacionales o lógicas.

6.1. Operadores aritméticos

Operador	Significado	Ejemplo
+	suma	$a + b$
-	resta / cambio de signo	$a - b$
*	multiplicación	precio * unidades
/	división	total / cantidad
MOD	resto de división entera	$n \text{ MOD } 2$
^	potencia	base ^ exponente

El operador MOD devuelve el resto de una división entera. Resulta especialmente útil para comprobar si un número es par o impar y para resolver problemas relacionados con ciclos, agrupaciones o repartos.

6.2. Operadores relacionales y lógicos

Grupo	Operadores	Ejemplo
Relacionales	=, <>, <, >, <=, >=	edad >= 18
Lógicos	Y, O, NO	edad >= 18 Y tienePermiso = Verdadero

6.3. Prioridad y paréntesis

Cuando una expresión contiene varios operadores, los paréntesis permiten hacer explícito el orden de evaluación. En ausencia de paréntesis, se aplican las reglas de prioridad de los operadores. Aunque conozcas esas reglas, utiliza paréntesis cuando mejoren la claridad y reduzcan el riesgo de interpretar mal una expresión.

```
media <- (nota1 + nota2 + nota3) / 3;
```

7. Operaciones básicas: Escribir, Leer y asignar

7.1. Escribir: salida

Escribir muestra en pantalla mensajes, valores de variables o resultados de expresiones. Es una de las primeras instrucciones que conviene dominar porque permite observar de forma inmediata qué información produce el algoritmo.

```
Algoritmo Presentacion
  Escribir "Hola";
  Escribir "Estoy aprendiendo algoritmos";
FinAlgoritmo
```

7.2. Leer: entrada

Leer detiene temporalmente la ejecución y espera a que el usuario introduzca un valor, que se almacena en la variable indicada. Antes de cada lectura conviene mostrar un mensaje claro que explique qué dato se espera y, cuando sea necesario, qué rango o formato debe respetar.

```
Algoritmo Saludar
  Definir nombre Como Caracter;
  Escribir "¿Cómo te llamas?";
  Leer nombre;
  Escribir "Hola ", nombre;
FinAlgoritmo
```

7.3. Asignación

```
resultado <- a + b;
doble <- numero * 2;
resto <- numero MOD 2;
```

La asignación evalúa primero la expresión situada a la derecha del operador <- y guarda el resultado en la variable situada a la izquierda. Si la variable ya contenía un valor, este queda sustituido por el nuevo.

7.4. Primer programa completo

```
Algoritmo SumaDosNumeros
  Definir a, b, suma Como Real;
  Escribir "Primer número:";
  Leer a;
  Escribir "Segundo número:";
  Leer b;
  suma <- a + b;
  Escribir "La suma es: ", suma;
FinAlgoritmo
```

Prueba mínima. Ejecuta el algoritmo con 3 y 4, con 0 y 0, y con un valor negativo. Comprobar varios casos forma parte de programar.

8. Decisiones: Si, SiNo y Segun

Una estructura alternativa permite que el algoritmo siga caminos diferentes según el resultado de una condición. Toda condición debe poder evaluarse como Verdadero o Falso.

8.1. Condicional simple

```
Si nota >= 5 Entonces
  Escribir "Aprobado";
FinSi
```

El bloque comprendido entre Si y FinSi se ejecuta únicamente cuando la condición es verdadera. Si la condición es falsa, ese bloque se omite y la ejecución continúa en la primera instrucción situada después de FinSi.

8.2. Condicional doble

```
Si edad >= 18 Entonces
    Escribir "Mayor de edad";
SiNo
    Escribir "Menor de edad";
FinSi
```

En una alternativa doble se ejecuta exactamente uno de los dos bloques: el asociado a la condición verdadera o el situado tras SiNo cuando la condición es falsa.

8.3. Condiciones anidadas

```
Si n = 0 Entonces
    Escribir "Cero";
SiNo
    Si n > 0 Entonces
        Escribir "Positivo";
    SiNo
        Escribir "Negativo";
    FinSi
FinSi
```

Es posible anidar decisiones, es decir, colocar una estructura Si dentro de otra. Sin embargo, un número excesivo de niveles dificulta la lectura. Utiliza sangría y busca una organización sencilla siempre que sea posible.

8.4. Selección múltiple con Segun

```
Segun opcion Hacer
    1: Escribir "Alta";
    2: Escribir "Consulta";
    3: Escribir "Salir";
De Otro Modo:
    Escribir "Opción incorrecta";
FinSegun
```

Segun resulta apropiado cuando una misma variable puede tomar varios valores concretos y cada valor debe activar un bloque diferente. De Otro Modo permite tratar cualquier valor que no coincida con los casos anteriores.

9. Repeticiones: Mientras, Repetir y Para

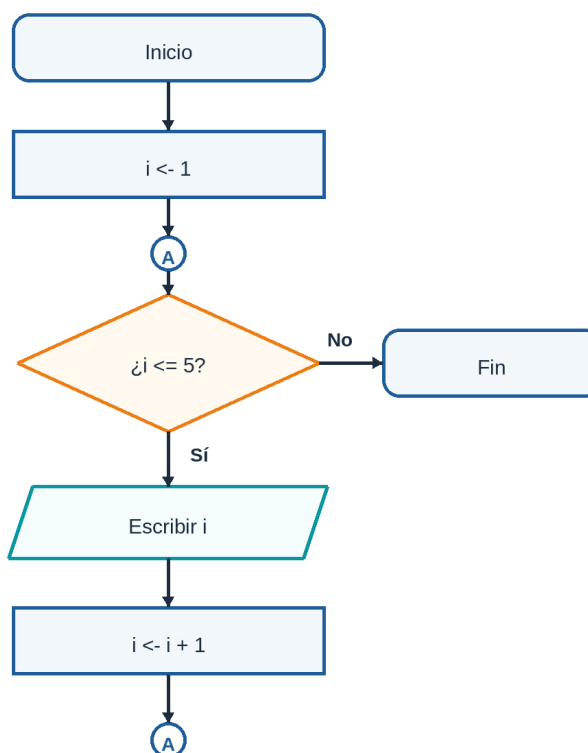
Los bucles permiten repetir un bloque de instrucciones de forma controlada. La elección de la estructura depende de la información que tengamos: si la condición debe comprobarse antes de ejecutar el cuerpo, si el bloque debe ejecutarse al menos una vez o si conocemos de antemano el número de repeticiones.

9.1. Mientras

Mientras es un bucle de precondición: evalúa la condición antes de ejecutar el cuerpo. Por tanto, si la condición es falsa desde el principio, las instrucciones del bucle no se ejecutan ninguna vez. Después de cada iteración, el flujo vuelve a la condición para decidir si debe repetirse el bloque.

```
Definir i Como Entero;
i <- 1;
Mientras i <= 5 Hacer
    Escribir i;
    i <- i + 1;
FinMientras
```

Bucle Mientras: la condición se evalúa antes



A = continuidad del flujo: al terminar el cuerpo, se vuelve a evaluar la condición.

Figura 2. Ordinograma de un bucle Mientras. La condición se evalúa antes de cada iteración. El conector A enlaza el final del cuerpo con una nueva evaluación de la condición; la rama No abandona el bucle.

9.2. Repetir ... Hasta Que

Repetir...Hasta Que es un bucle de poscondición: primero ejecuta el cuerpo y después evalúa la condición de salida. Por ello, las instrucciones del bloque se ejecutan al menos una vez.

```

Repetir
  Escribir "Introduce una nota entre 0 y 10:";
  Leer nota;
Hasta Que nota >= 0 Y nota <= 10

```

Repetir...Hasta Que: el cuerpo se ejecuta al menos una vez

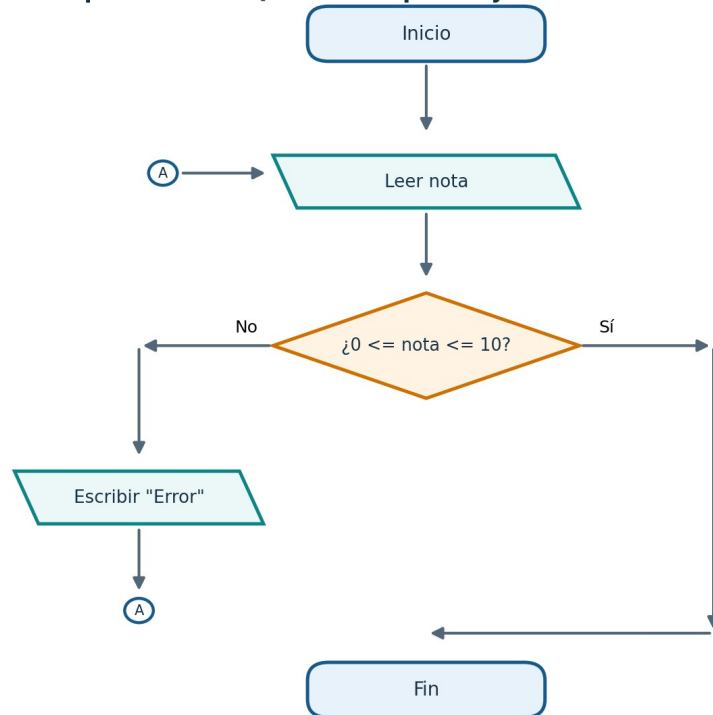


Figura 3. Ordinograma de una validación con Repetir...Hasta Que. El dato se solicita antes de comprobar si cumple la condición de salida.

9.3. Para

Para es especialmente adecuado cuando conocemos de antemano el número de repeticiones o podemos expresarlo mediante un contador con valor inicial, valor final e incremento. El paso puede ser distinto de 1 cuando el problema lo requiera.

```

Para i <- 1 Hasta 10 Con Paso 1 Hacer
  Escribir i;
FinPara

```

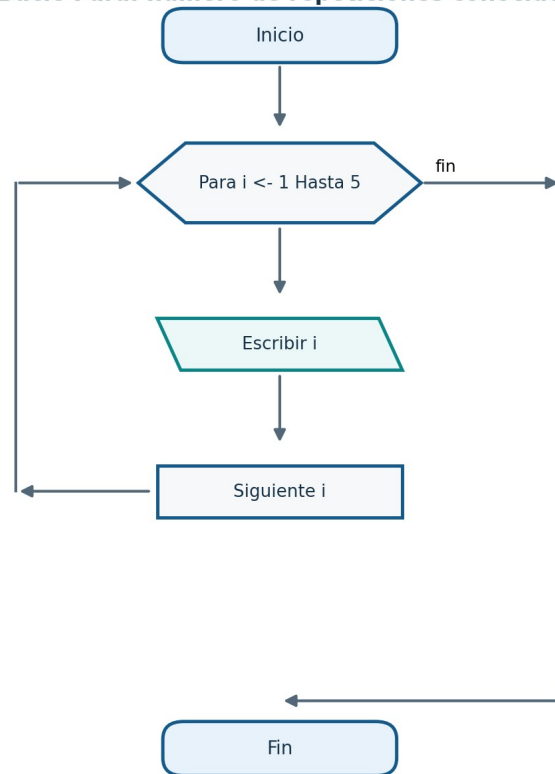
Bucle Para: número de repeticiones conocido

Figura 4. Representación didáctica de un bucle contado Para mediante el símbolo de repetición definida.

9.4. ¿Qué bucle elegir?

Situación	Bucle recomendado
No sabemos si habrá alguna repetición	Mientras
Debe ejecutarse al menos una vez	Repetir ... Hasta Que
Conocemos la cantidad o un contador claro	Para

Peligro clásico. Todo bucle condicional debe modificar algún dato relacionado con su condición; de lo contrario puede convertirse en un bucle infinito.

10. Contadores, acumuladores y banderas

Al trabajar con bucles aparecen patrones que se repiten en muchos problemas. Reconocerlos permite diseñar soluciones con mayor rapidez y evita empezar desde cero cada vez.

10.1. Contador

Un contador registra cuántas veces ocurre un suceso. Habitualmente se inicializa a cero y se incrementa cuando se produce el evento que queremos contabilizar.

```
contador <- 0;
Si nota >= 5 Entonces
    contador <- contador + 1;
FinSi
```

10.2. Acumulador

Un acumulador conserva una cantidad que va creciendo o cambiando con valores sucesivos. Debe inicializarse antes de comenzar el proceso repetitivo y actualizarse dentro del bucle.

```
suma <- 0;
Para i <- 1 Hasta 5 Hacer
    Leer numero;
    suma <- suma + numero;
FinPara
```

10.3. Bandera o indicador lógico

Una bandera, también llamada indicador lógico, almacena un estado Verdadero/Falso. Se utiliza para recordar si ha ocurrido una determinada situación durante la ejecución.

```
encontrado <- Falso;
Si numero = buscado Entonces
    encontrado <- Verdadero;
FinSi
```

10.4. Máximo provisional

Para obtener el mayor de varios valores necesitamos conservar el mejor valor encontrado hasta ese momento. Una estrategia segura consiste en inicializar la variable mayor con el primer dato válido y comparar cada valor posterior con ella.

11. Diagramas de flujo y ordinogramas

Un diagrama de flujo representa de forma visual la secuencia de ejecución de un algoritmo. Cuando el diagrama describe con detalle las operaciones de un programa se denomina ordinograma. Su utilidad principal es hacer visibles el orden de las acciones, las decisiones, las repeticiones y los puntos en los que distintos caminos vuelven a unirse.

11.1. Organigrama y ordinograma

En sentido amplio, un diagrama de flujo puede representar procesos o sistemas de distinta naturaleza. En programación trabajaremos con ordinogramas, centrados en la lógica detallada de un algoritmo. La representación debe poder comprenderse con independencia del lenguaje de programación utilizado.

11.2. Símbolos fundamentales

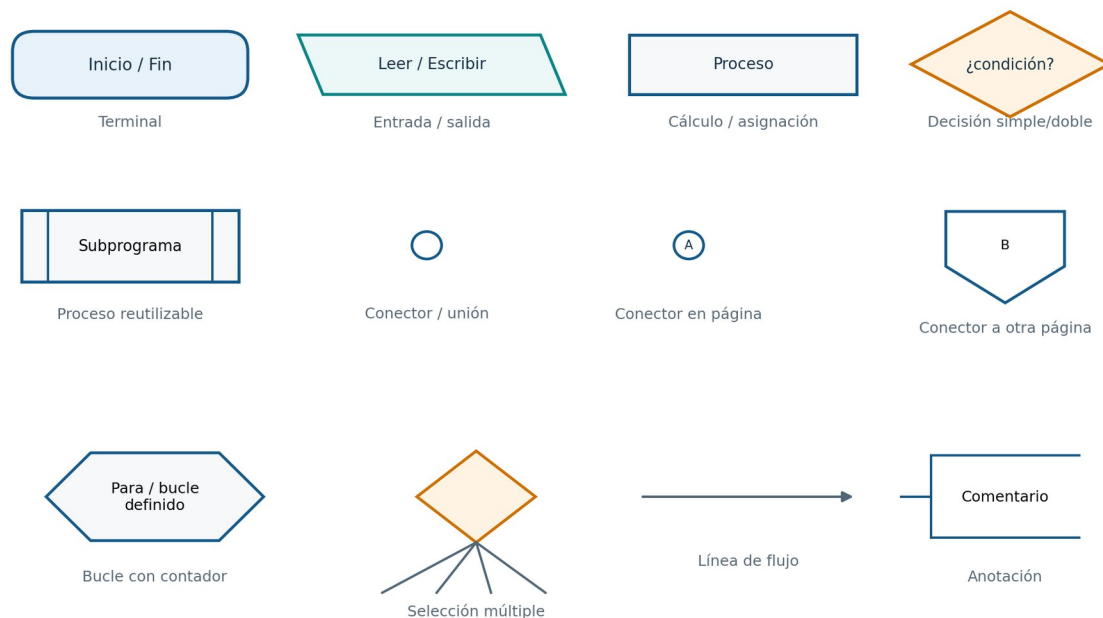


Figura 5. Símbolos de trabajo para los ordinogramas de la asignatura.

Símbolo	Uso
Terminal	inicio, fin o parada del algoritmo
Entrada / salida	lectura de datos o presentación de resultados
Proceso	cálculos, asignaciones y transformaciones
Decisión	condición con dos caminos o selección
Subprograma	llamada a un bloque reutilizable
Conector circular	unir líneas de flujo, enlazar puntos o evitar cruces
Conector a otra página	continuar el diagrama en otra página

Símbolo	Uso
Bucle definido	representar una repetición con contador conocido
Línea de flujo	indicar el sentido de ejecución
Comentario	añadir una aclaración que no forma parte de la ejecución

11.3. El conector circular: cuándo y por qué usarlo

El conector circular cumple una función concreta: permite enlazar puntos del flujo, reunir caminos y evitar trazados largos o cruces de líneas. Cuando varias ramas de una decisión vuelven a una misma secuencia, puede utilizarse un conector circular para hacer explícita la unión. Si el flujo continúa en otro punto de la misma página, los conectores correspondientes pueden identificarse con la misma letra o número; si continúa en otra página, se emplea el símbolo específico de fuera de página.

Corrección importante. Las ramas no deben “pegarse” de forma ambigua a una línea horizontal. Cuando el esquema necesita un punto explícito de reunión, usaremos un conector circular, tal como se muestra en los ejemplos siguientes.

11.4. Reglas de construcción

- Todos los símbolos que forman parte del algoritmo deben quedar unidos mediante líneas de flujo.
- Las líneas de flujo no deben cruzarse. Si el trazado se vuelve complejo, utiliza conectores para mantener la representación clara.
- El sentido principal de lectura debe avanzar, preferentemente, de arriba abajo y, cuando sea posible, de izquierda a derecha.
- Etiqueta las salidas de las decisiones para indicar qué camino corresponde a cada resultado, por ejemplo Sí y No.
- Al símbolo de inicio no debe llegar ninguna línea de flujo. Del símbolo de fin no debe salir ninguna línea.
- De un símbolo de proceso sale una única línea de flujo normal. Cuando varios caminos deben confluir, haz explícito el punto de unión mediante un conector si ello mejora la claridad.
- Mantén una separación suficiente entre símbolos y procura que las ramas de decisiones y bucles conserven una disposición regular y legible.
- El ordinograma y el pseudocódigo deben representar exactamente la misma lógica. Si describen recorridos diferentes, al menos una de las dos representaciones contiene un error.

11.5. Ejemplo completo: signo de un número

El algoritmo lee un número y debe indicar si es cero, positivo o negativo. La primera decisión separa el caso cero. Cuando el valor no es cero, una segunda decisión distingue entre positivo y negativo. Después de escribir el resultado, los caminos vuelven a una secuencia común mediante conectores circulares antes de finalizar.

Ejemplo: clasificar un número por su signo

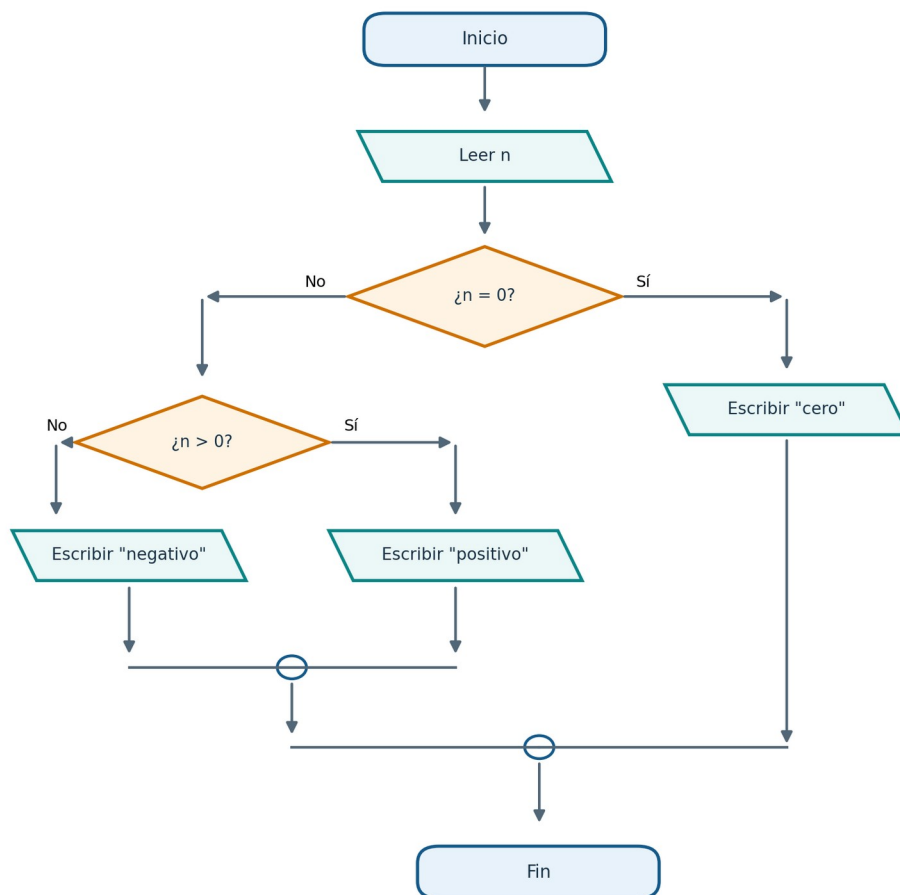


Figura 6. Ordinograma para clasificar un número según su signo. Los conectores circulares hacen explícitos los puntos en los que vuelven a reunirse las ramas.

```

Algoritmo Signo
  Definir n Como Real;
  Leer n;
  Si n = 0 Entonces
    Escribir "Cero";
  SiNo
    Si n > 0 Entonces
      Escribir "Positivo";
    SiNo
      Escribir "Negativo";
    FinSi
  FinSi
FinAlgoritmo
  
```

11.6. Ejemplo completo: mayor de dos números

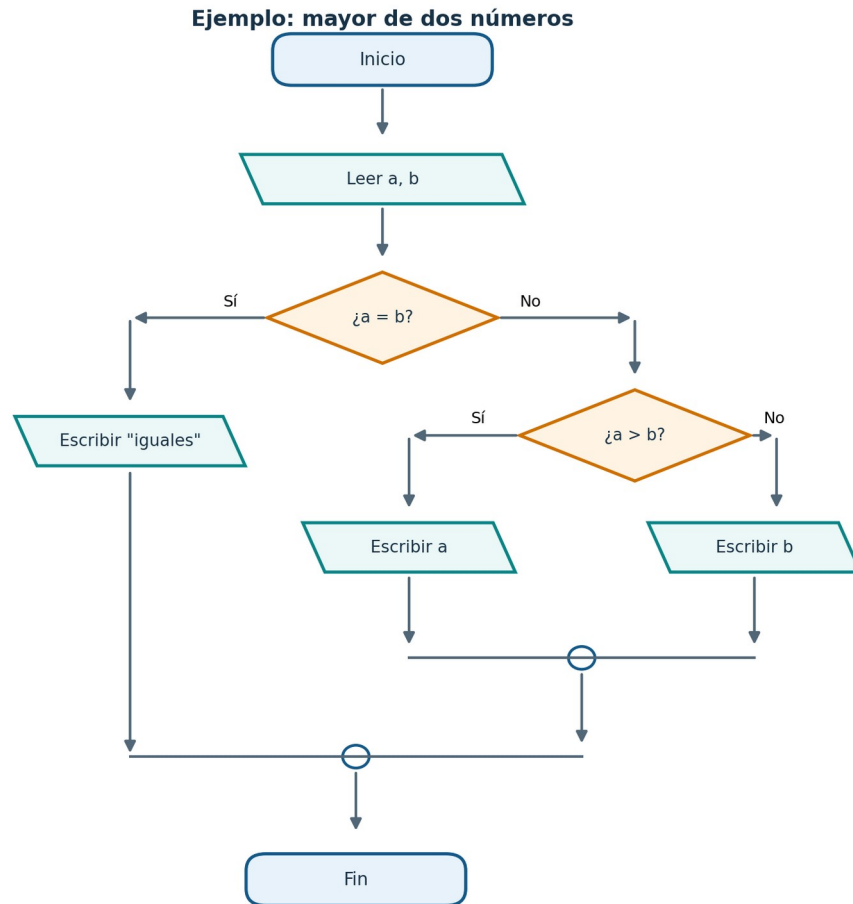


Figura 7. Ordinograma para comparar dos valores. Las distintas salidas de las decisiones confluyen mediante conectores circulares antes de finalizar.

```

Algoritmo MayorDeDos
  Definir a, b Como Real;
  Leer a, b;
  Si a = b Entonces
    Escribir "Son iguales";
  SiNo
    Si a > b Entonces
      Escribir a;
    SiNo
      Escribir b;
    FinSi
  FinSi
FinAlgoritmo
  
```

12. Trazas y depuración

Una traza consiste en simular manualmente la ejecución de un algoritmo, instrucción por instrucción, y anotar cómo cambia el valor de las variables. Es una técnica especialmente útil para comprender bucles y decisiones y para localizar errores lógicos.

12.1. Ejemplo de traza

```
suma <- 0;
Para i <- 1 Hasta 4 Hacer
    suma <- suma + i;
FinPara
Escribir suma;
```

Paso	i	suma antes	Operación	suma después
Inicial	-	-	suma <- 0	0
1	1	0	0 + 1	1
2	2	1	1 + 2	3
3	3	3	3 + 3	6
4	4	6	6 + 4	10

La tabla muestra que el resultado final es 10. Si el programa produjera un valor diferente, la traza permitiría localizar la primera iteración en la que el estado real deja de coincidir con el esperado.

12.2. Tipos de errores

Tipo	Qué ocurre	Ejemplo
Sintaxis	PSeInt no puede interpretar una instrucción.	Falta FinSi o una palabra clave está mal escrita.
Ejecución	El algoritmo intenta una operación problemática.	División entre cero.
Lógica	El algoritmo ejecuta, pero el resultado es incorrecto.	Condición > cuando debía ser >=.
Bucle infinito	La condición nunca deja de cumplirse.	Olvidar incrementar el contador.

12.3. Método de depuración

- Reproduce el error con un caso pequeño.
- Anota los valores iniciales de las variables.
- Recorre el algoritmo en orden y registra cada cambio.
- Comprueba en cada decisión qué rama debería tomarse.
- Corrige una causa cada vez y vuelve a ejecutar los mismos datos.

13. Ejemplos resueltos paso a paso

13.1. Par o impar

Idea: el resto de dividir un número entero entre 2 permite determinar su paridad. Si el resto es cero, el número es par; en caso contrario, es impar.

```
Algoritmo ParImpar
  Definir n Como Entero;
  Escribir "Introduce un entero:";
  Leer n;
  Si n MOD 2 = 0 Entonces
    Escribir "Par";
  SiNo
    Escribir "Impar";
  FinSi
FinAlgoritmo
```

Pruebas recomendadas: 0, 1, 2, -3 y 100.

13.2. Media de varias notas

Necesitamos un acumulador (suma), un contador (i) y la cantidad de notas (n). Las notas se leen y acumulan dentro del bucle; la media se calcula una sola vez, después de finalizar la repetición.

Antes de dividir se comprueba que n sea mayor que cero.

```
Algoritmo MediaNotas
  Definir n, i Como Entero;
  Definir nota, suma, media Como Real;
  Escribir "¿Cuántas notas?";
  Leer n;
  suma <- 0;
  Para i <- 1 Hasta n Hacer
    Escribir "Nota ", i, ":";
    Leer nota;
    suma <- suma + nota;
  FinPara
  Si n > 0 Entonces
    media <- suma / n;
    Escribir "Media: ", media;
  SiNo
    Escribir "No hay notas";
  FinSi
FinAlgoritmo
```

13.3. Validar una entrada

Cuando el usuario debe introducir una nota válida, Repetir...Hasta Que resulta natural porque primero necesitamos solicitar el dato y solo después podemos comprobar si pertenece al intervalo permitido.

```
Algoritmo NotaValida
  Definir nota Como Real;
  Repetir
    Escribir "Nota [0..10]:";
    Leer nota;
    Si nota < 0 O nota > 10 Entonces
      Escribir "Valor incorrecto";
    FinSi
  Hasta Que nota >= 0 Y nota <= 10
  Escribir "Nota aceptada: ", nota;
FinAlgoritmo
```

13.4. Tabla de multiplicar

```

Algoritmo TablaMultiplicar
  Definir n, i Como Entero;
  Leer n;
  Para i <- 1 Hasta 10 Hacer
    Escribir n, " x ", i, " = ", n * i;
  FinPara
FinAlgoritmo

```

13.5. Adivinar un número

Este ejemplo combina generación aleatoria, repetición y decisiones. En cada intento se indica si el número secreto es mayor o menor que el valor introducido. El bucle finaliza únicamente cuando ambos valores coinciden.

```

Algoritmo AdivinarNumero
  Definir secreto, intento Como Entero;
  secreto <- Aleatorio(1, 10);
  Repetir
    Escribir "Adivina el número (1-10):";
    Leer intento;
    Si intento < secreto Entonces
      Escribir "El número secreto es mayor";
    SiNo
      Si intento > secreto Entonces
        Escribir "El número secreto es menor";
      FinSi
    FinSi
  Hasta Que intento = secreto
  Escribir "Correcto";
FinAlgoritmo

```

Ampliación. Añade un contador de intentos y muestra cuántas oportunidades ha necesitado el jugador.

14. Banco de ejercicios

Los ejercicios están ordenados por dificultad. Antes de ejecutar cada solución, anota qué esperas que ocurra con al menos dos casos de prueba. Después compara esa predicción con el resultado real y corrige el algoritmo si existe alguna diferencia.

A. Escribir y expresiones

- Muestra tu nombre, grupo y una frase relacionada con la programación.
- Calcula y muestra el resultado de $25 + 17$.
- Muestra el área de un rectángulo con base 8 y altura 5.
- Muestra el resultado de $17 \text{ MOD } 5$.

B. Leer y calcular

- Pide dos números y muestra suma, resta y multiplicación.
- Pide el radio de un círculo y calcula su área.
- Pide grados Celsius y conviértelos a Fahrenheit.
- Pide precio y unidades y calcula el total.

C. Condicionales

- Pide una edad y muestra si es mayor de edad.

- Pide un número e indica si es positivo, negativo o cero.
- Pide dos números y muestra el mayor o indica que son iguales.
- Pide una nota y clasifícala como suspenso o aprobado.
- Pide un año y decide si es múltiplo de 4 como primera aproximación a un año bisiesto.

D. Mientras

- Muestra los números del 1 al 10.
- Suma los enteros del 1 a N.
- Pide una contraseña hasta que sea correcta.
- Lee números hasta que se introduzca un cero y cuenta cuántos se han leído.

E. Repetir

- Pide una nota hasta que esté entre 0 y 10.
- Pide una opción de menú hasta elegir salir.
- Solicita un número positivo y rechaza cero o negativos.

F. Para

- Muestra una tabla de multiplicar.
- Muestra los pares entre 2 y 100.
- Calcula el factorial de N.
- Lee cinco números y muestra su suma.

G. Retos

- Lee cinco números y determina el mayor sin usar vectores.
- Juego de adivinar número con contador de intentos.
- Calcula cuántos aprobados hay entre N notas y su media.
- Genera un menú que permita repetir operaciones hasta elegir salir.

Criterio de entrega. Una solución no está terminada solo porque “ejecuta”: debe usar nombres claros, estar bien sangrada, terminar correctamente y haber sido probada con varios datos.

15. Referencia rápida

15.1. Plantilla mínima

```
Algoritmo Nombre
  // Declaraciones

  // Instrucciones
FinAlgoritmo
```

15.2. Plantillas de estructuras

Estructura	Plantilla
Si	Si condicion Entonces instrucciones FinSi
Si / SiNo	Si condicion Entonces bloque A SiNo bloque B FinSi
Mientras	Mientras condicion Hacer instrucciones FinMientras
Repetir	Repetir instrucciones Hasta Que condicion
Para	Para i <- inicio Hasta fin Con Paso 1 Hacer instrucciones FinPara

15.3. Checklist antes de dar un ejercicio por terminado

- ¿El algoritmo empieza y termina correctamente?
- ¿Todas las variables están declaradas con un tipo coherente?
- ¿Los mensajes de entrada explican qué debe introducir el usuario?
- ¿Las asignaciones usan <- y las condiciones usan operadores de comparación?
- ¿Cada Si, Mientras y Para está correctamente cerrado?
- ¿Los bucles cambian los datos necesarios para terminar?
- ¿Has probado valores normales, límite y algún caso inesperado?
- Si has dibujado un ordinograma, ¿sus ramas y uniones coinciden exactamente con el pseudocódigo?
- ¿Los puntos de unión complejos usan conectores circulares y no líneas cruzadas?

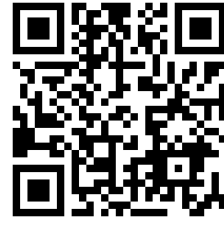
15.4. Recursos del curso

[Web de ejercicios](#)

[PSeInt Web](#)



<https://rubenperezgva.github.io/pseint-1daw/>



<https://www.pseint-web.app/>

Idea final. PSeInt nos ayuda a pensar con estructuras que después reconoceremos en Java: variables, expresiones, condicionales, bucles y descomposición del problema. La sintaxis cambiará; la lógica permanece.

FIN DE LA GUÍA

Material didáctico elaborado por Rubén Pérez Ibáñez para el módulo de Programación de 1.º DAW · IES Álvaro Falomir · Curso 2026/27.